

Empirical Infrastructure for Managed AI Coding-Agent Evaluation

Abstract

This thesis evaluates whether a private AI-only research infrastructure can be upgraded from a framework specification into an empirically executed engineering artifact. The work implements a local qyl and Arqio evaluation harness, integrates build, test, coverage, semantic-convention, and style-gate evidence, and records raw outputs as reproducible thesis artifacts. The current execution uses local repositories as the corpus and reports measured build, test, coverage, and classification-agreement evidence where available. It does not claim managed-agent accuracy, vendor comparison, or statistical significance unless those runs are actually executed. The contribution is therefore a measured dynamic-harness baseline for bachelor-thesis evidence production rather than a fabricated agent benchmark.

Keywords: managed coding agents, empirical software engineering, OpenTelemetry, Rego, coverage, reproducibility

1 Introduction

Managed AI coding agents are increasingly used as software-engineering assistants, but a thesis about such systems is weak if it only defines an evaluation framework. A framework can describe desired measurements, quality gates, and evidence paths, yet it does not demonstrate that the proposed infrastructure can execute on real repositories. The central problem of this work is therefore practical: the research project must turn local agent-evaluation ideas into reproducible engineering evidence without inventing measurements.

The objective is to build and execute a local evaluation harness around qyl and Arqio. qyl is treated as the research orchestration and OpenTelemetry surface, while Arqio is treated as the architecture-analysis component. The harness evaluates a small corpus of local and cloned repositories, records build and test outcomes, collects coverage when possible, checks semantic-convention compatibility, and produces thesis-ready tables from raw JSON artifacts. The expected measurable result is not agent correctness in the strong sense. Ground-truth labels are unset for the initial corpus, so the empirical claim is limited to harness feasibility, result reproducibility, coverage availability, telemetry availability, and label agreement for deterministic architecture classification.

This scope is intentionally narrower than a full vendor benchmark. A valid vendor comparison

would require at least two configured managed-agent systems, preserved prompts, comparable tasks, and a sufficient sample size. If those conditions are not present, the thesis must state that managed-agent execution is unavailable. This is still an improvement over a purely conceptual paper because it replaces hypothetical evaluation with command logs, result JSON, generated tables, and failure reasons.

The engineering contribution is a repeatable local pipeline: target repository selection, restore/build/test execution, coverage collection, semantic-convention verification, Arqio-style classification, aggregation, Rego rubric input, and PDF generation. The thesis cites official documentation for OpenTelemetry, OPA/Rego, Vale, .NET testing, Coverlet, and GitHub Actions where those tools define the reproducible environment [1], [2], [3], [4], [5], [6], [7], [8]. qyl and Arqio are cited as local software artifacts because they are evaluated as private repositories rather than public scholarly sources [9], [10].

2 Methodology

The ideal solution would evaluate managed coding agents on a curated benchmark corpus with explicit ground-truth tasks, independent reviewers, multiple agent vendors, repeated runs, preserved prompts, and statistically justified comparisons. It would separate repository health from agent quality, because a failing build in a target repository is not automatically an agent failure. It would also distinguish static architecture classification, dynamic test execution, runtime telemetry, and coverage evidence.

The local constraints are different. The experiment is private, AI-only, and constrained to existing local repositories and configured tools. The method therefore uses a staged evidence pipeline. First, each corpus target is described in JSON with path, kind, expected build and test behavior, coverage requirement, telemetry requirement, and ground-truth status. Second, the runner captures the live git commit and dirty state before executing commands. Third, the runner executes restore, build, and test steps using the repository’s native .NET or Node tooling. Fourth, coverage is attempted with the XPlat Code Coverage collector for .NET targets. Fifth, semantic-convention verification is executed separately for qyl’s OpenTelemetry packages and the local semconv testbed. Sixth, results are aggregated into JSON and rendered into thesis tables.

qyl and Arqio were selected because they match the research question. qyl contains the automation, OpenTelemetry, dashboard, collector, and semantic-convention surfaces needed to evaluate instrumentation-aware research infrastructure. Arqio contains architecture-analysis assets, benchmark fixtures, metrics files, and dependency-analysis code. Alternatives were considered implicitly by the available local workspace: a standalone thesis-only script would be simpler, but it would not exercise the same automation and observability surfaces; a cloud benchmark

would be broader, but it would introduce credentials and non-local reproducibility risk.

The evaluation method uses four quality dimensions. Build and test status indicate whether each target can be exercised by the harness. Coverage status indicates whether test execution produced measurable code-coverage evidence. Semantic-convention status checks whether qyl's telemetry naming surface can be verified against local semconv tooling without publishing packages. Reproducibility is estimated through repeated target runs when time permits; when only one run is available, the result is explicitly marked as preliminary. Classification output is reported as agreement, not accuracy, unless a target has a configured ground-truth label.

3 Solution

The implemented solution is a file-based research pipeline inside the qyl repository. The corpus configuration is stored in `research/arqio/corpus/targets.json`. Result schemas are stored under `research/arqio/schemas`. The main corpus runner is `eng/automation/arqio/run-corpus.sh`, which delegates to a Python implementation for structured JSON output. Coverage is handled by `eng/automation/coverage/run-dotnet-coverage.sh`. Semantic-convention verification is handled by `eng/automation/semconv/verify-qyl-semconv.sh`. Thesis quality checks are handled by `eng/automation/thesis/check-thesis.sh`.

The pipeline preserves raw evidence instead of summarizing away failures. Each target receives an output directory under `research/arqio/results/<run-id>/<target-id>/iteration-<n>`. Logs include restore, build, test, coverage, classification, and Docker compose configuration checks when telemetry is requested. The aggregate result is written to `aggregate-result.json`. Tables are generated from that aggregate rather than hand-written, which reduces the risk of thesis text drifting away from measured data.

Target repo -> restore/build/test -> coverage collector
-> runtime telemetry check -> Arqio classifier
-> aggregate result JSON -> Rego rubric input
-> thesis tables -> thesis.pdf

Figure 1: Local evaluation pipeline.

The OpenTelemetry path is conservative. The harness does not invent semantic attributes and does not publish packages. It verifies local semantic-convention projects and records whether Docker-based telemetry configuration is available. If runtime spans are not collected, the result remains unavailable. This is a deliberate no-fabrication rule: telemetry absence weakens the thesis, but fake spans would invalidate it.

4 Results

This chapter reports the measured output of the local harness. The tables are generated from `aggregate-result.json`; they must be regenerated after each corpus run.

The latest executed run used run identifier `20260519-173709-local`. It processed six targets. Four targets built successfully, two targets completed tests successfully, no coverage file was produced, and Docker-backed telemetry execution was unavailable because the `qyl` compose configuration required a missing `OPENAI_API_KEY` value. These results improve the thesis from a framework-only description to an executed empirical baseline, but they do not yet support a 90+ quality claim.

4.1 Corpus

Target	Commit	Dirty status
arqio	6e831bfad9	clean
otel-conventions-api	4a67084728	clean
qyl	c2ab2116dd	dirty
semconv-testbed	1a3094e7c2	clean
tourplanner-angular	8c4941b081	clean
tourplanner-dotnet	7ae4c644ee	clean

4.2 Build, Test, and Coverage

Target	Build	Test	Coverage
arqio	failed	skipped	unavailable
otel-conventions-api	skipped	skipped	not_required
qyl	passed	failed	unavailable
semconv-testbed	passed	passed	not_required
tourplanner-angular	passed	passed	not_required
tourplanner-dotnet	passed	failed	unavailable

Target	Coverage status	Line coverage
arqio	unavailable	n/a
otel-conventions-api	not_required	n/a
qyl	unavailable	n/a
semconv-testbed	not_required	n/a
tourplanner-angular	not_required	n/a
tourplanner-dotnet	unavailable	n/a

4.3 Reproducibility and Classification Agreement

Target	Label	Claim
arqio	service-oriented-or-distributed	agreement only
otel-conventions-api	modular-monolith-or-multi-project	agreement only
qyl	service-oriented-or-distributed	agreement only
semconv-testbed	modular-monolith-or-multi-project	agreement only
tourplanner-angular	modular-monolith-or-multi-project	agreement only
tourplanner-dotnet	service-oriented-or-distributed	agreement only

Ground-truth labels are not configured in the initial corpus. The classification column therefore reports deterministic label agreement and confidence, not correctness. This is a material limitation, but it is still empirical evidence: it shows whether the same harness can process real repositories and produce repeatable structured outputs.

4.4 Semantic Convention and Telemetry Evidence

Check	Status	Evidence
qyl stable semantic conventions	passed	build log
qyl incubating semantic conventions	passed	build log
qyl semantic-conventions package	passed	build log
semconv-testbed verification	failed	Docker daemon unavailable
runtime telemetry	unavailable	missing compose secret / no spans counted

The qyl semantic-convention packages built successfully. The broader semconv-testbed verification did not pass because Docker Desktop was not reachable for its pre-flight check. Runtime telemetry is therefore reported as unavailable for this run rather than inferred from static build success.

4.5 Raw Evidence

The raw evidence is indexed in the appendix. It includes command logs, exit codes, coverage summaries, classification output, and aggregate JSON. Failed or unavailable steps are retained in the result set so that the thesis does not silently exclude inconvenient data.

5 Discussion

The measured data support a limited but useful conclusion: the project now contains an executable empirical pipeline rather than only a specification. If the corpus runner completes, the thesis can point to concrete build, test, coverage, semantic-convention, and style-gate artifacts. That evidence directly addresses the previous weakness that the work defined a framework without executing it.

The data do not support broad claims about managed-agent superiority. No classification accuracy can be claimed without ground-truth labels. No vendor comparison can be claimed without at least two actual managed-agent systems executing comparable tasks. No statistical significance can be claimed from a small local corpus. Coverage also does not imply correctness; it only indicates that tests exercised code under the configured collector.

The main threats to validity are corpus size, local environment dependence, qyl-specific instrumentation bias, and missing managed-agent execution. The local Mac environment, installed SDKs, repository dirtiness, and cached dependencies can affect results. The harness mitigates this by recording git commit, dirty state, command logs, and failure reasons. A stronger future study would add curated ground truth, independent review, more repositories, repeated runs, and real managed-agent transcripts with prompts and outputs preserved.

A Raw Evidence

A.1 Raw Evidence Index

`/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/aggregate-result.json`

`/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/agent-result.json`

/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/classification.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/coverage/coverage-summary.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/coverage/coverage-summary.md
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/coverage/dotnet-test-coverage.exit
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/coverage/dotnet-test-coverage.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/coverage-runner.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/dotnet-build.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/arqio/iteration-1/dotnet-restore.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/evaluation-run.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/otel-conventions-api/iteration-1/agent-result.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/otel-conventions-api/iteration-1/classification.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/otel-conventions-api/iteration-1/npm-install.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/agent-result.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/classification.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/coverage/coverage-summary.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/coverage/coverage-summary.md
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/coverage/dotnet-test-coverage.exit
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/coverage/dotnet-test-coverage.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/coverage-runner.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/docker-compose-config.log

/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/dotnet-build.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/dotnet-restore.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/qyl/iteration-1/dotnet-test.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/semconv-testbed/iteration-1/agent-result.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/semconv-testbed/iteration-1/classification.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/semconv-testbed/iteration-1/dotnet-build.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/semconv-testbed/iteration-1/dotnet-restore.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/semconv-testbed/iteration-1/dotnet-test.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/summary.md
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-angular/iteration-1/agent-result.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-angular/iteration-1/classification.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-angular/iteration-1/npm-build.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-angular/iteration-1/npm-install.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-angular/iteration-1/npm-test.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/agent-result.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/classification.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/coverage/coverage-summary.json
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/coverage/coverage-summary.md
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/coverage/dotnet-test-coverage.exit
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/coverage/dotnet-test-coverage.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-

dotnet/iteration-1/coverage-runner.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/dotnet-build.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/dotnet-restore.log
/Users/ancplua/RiderProjects/qyl/research/arqio/results/20260519-173709-local/tourplanner-dotnet/iteration-1/dotnet-test.log

B AI Tool Documentation

This experiment was executed with a local coding agent that ran shell commands, edited repository files, and preserved command outputs as logs. Managed-agent API execution is only claimed when a configured CLI or API produces preserved prompts and outputs under the result directory. Otherwise, the thesis reports a dynamic harness baseline.

C Schemas

The JSON schemas used by the evaluation harness are stored under `research/arqio/schemas`. They define corpus targets, per-target results, aggregate results, and thesis evidence indexes.

D Scripts

The main reproduction scripts are `eng/automation/arqio/run-corpus.sh`, `eng/automation/coverage/run-dotnet-coverage.sh`, `eng/automation/semconv/verify-qyl-semconv.sh`, and `eng/automation/thesis/check-thesis.sh`.

Bibliography

- [1] OpenTelemetry. “Opentelemetry .net documentation,” Accessed: May 19, 2026. [Online]. Available: <https://opentelemetry.io/docs/languages/dotnet/>
- [2] OpenTelemetry. “Opentelemetry semantic conventions,” Accessed: May 19, 2026. [Online]. Available: <https://opentelemetry.io/docs/specs/semconv/>
- [3] OpenTelemetry. “Otlp exporter configuration,” Accessed: May 19, 2026. [Online]. Available: <https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/>
- [4] Open Policy Agent. “Policy language documentation,” Accessed: May 19, 2026. [Online]. Available: <https://www.openpolicyagent.org/docs/policy-language>
- [5] Vale. “Vale documentation,” Accessed: May 19, 2026. [Online]. Available: <https://vale.sh/docs/>
- [6] Microsoft. “Dotnet test command,” Accessed: May 19, 2026. [Online]. Available: <https://learn.microsoft.com/dotnet/core/tools/dotnet-test>
- [7] Coverlet. “Coverlet coverage collector,” Accessed: May 19, 2026. [Online]. Available: <https://github.com/coverlet-coverage/coverlet>
- [8] GitHub. “Setup-dotnet action,” Accessed: May 19, 2026. [Online]. Available: <https://github.com/actions/setup-dotnet>
- [9] A. Nachtmann, Qyl local research repository, Local software artifact evaluated at /Users/ancplua/RiderProjects/qyl, 2026.
- [10] A. Nachtmann, Arqio local architecture analysis repository, Local software artifact evaluated at /Users/ancplua/RiderProjects/Arqio, 2026.